ML Lab — Plano de arquitetura do backend

Oct 22, 2026 · @Simão

Visão geral

O ML Lab é um backend em Python onde se carrega um dataset, se escolhe um tipo de aprendizagem e um algoritmo, e se obtém o treino passo a passo (snapshots), as métricas de avaliação e o registo da experiência. A interface visual fica para a última fase; até lá, tudo é usado por código, testes e uma API.

Objetivos

1. Aprender ML: cada algoritmo expõe o que acontece durante o treino, não só o resultado final.
2. Aprender as tecnologias: cada biblioteca tem um papel único e claro (secção Tecnologias).
3. Manutenção fácil: adicionar um algoritmo, uma métrica ou um tipo de tarefa = criar um ficheiro e registá-lo, sem alterar código existente.

Princípios de arquitetura

- **Núcleo independente da interface.** O pacote `core` não importa FastAPI, nem bibliotecas de gráficos, nem nada de UI. A API e a futura interface consomem o núcleo, nunca o contrário.
- **Contratos antes de implementações.** Modelos, avaliadores, transformações e fontes de dados obedecem a interfaces abstratas. O resto do sistema só conhece a interface.
- **Registo (plugins).** Cada componente regista-se sozinho num registo central com um decorador. Nada de `if algoritmo == "knn"` espalhados pelo código.
- **Configuração declarativa e validada.** Hiperparâmetros e experiências são descritos por esquemas Pydantic: validados à entrada, serializáveis para JSON e reutilizáveis pela UI para gerar formulários.
- **Observabilidade por eventos.** O treino emite eventos (início, passo, snapshot, fim). Quem precisa deles subscreve-os: registo, API em tempo real, testes.
- **Reprodutibilidade.** Toda a aleatoriedade passa por uma seed explícita; cada experiência guarda dataset, versão do código, configuração e seed.
- **Dependências apontam para dentro.** Camadas externas dependem das internas, nunca o inverso (ver Arquitetura em camadas).

Decisões assumidas (a confirmar)

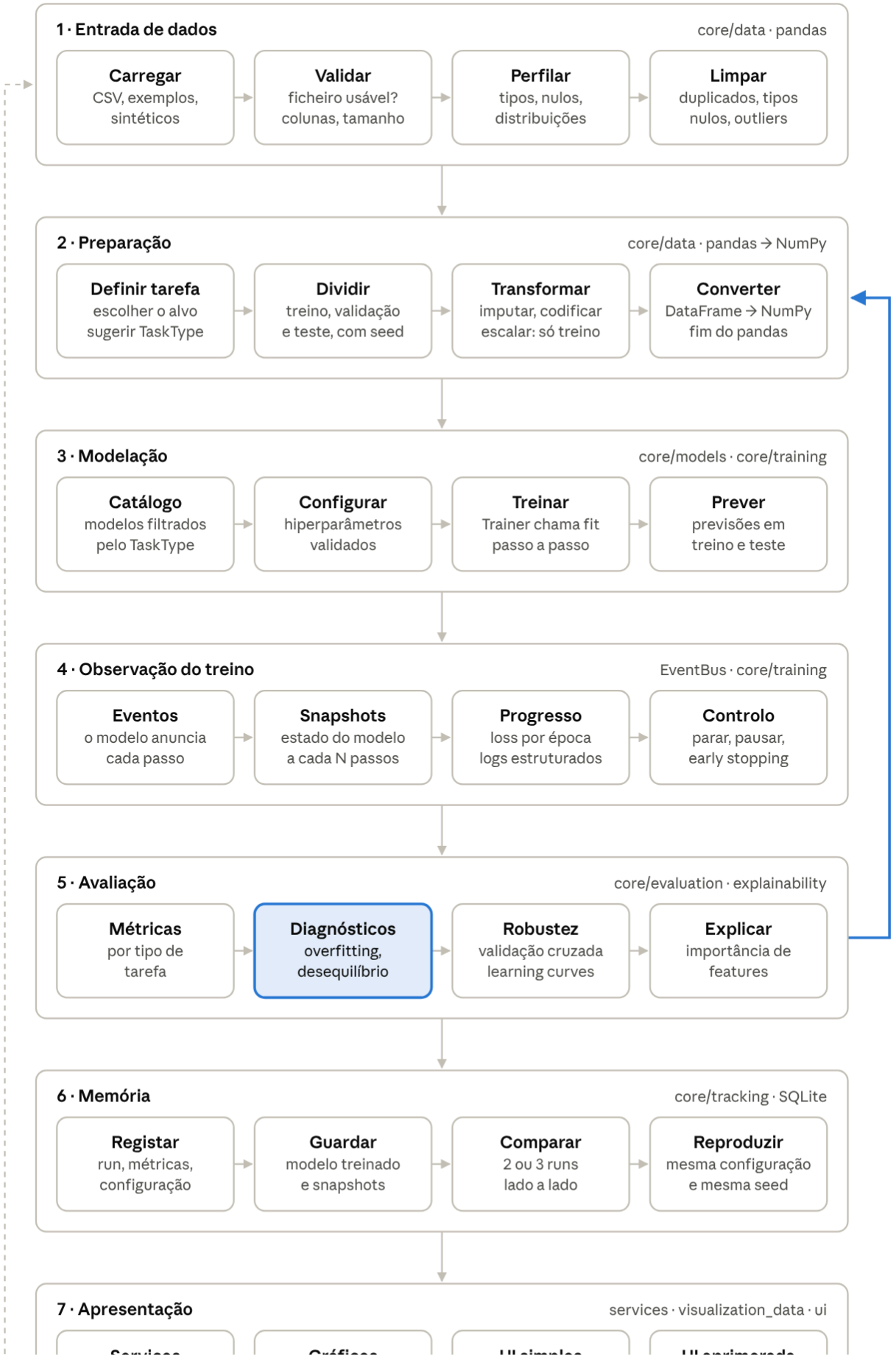
- Stack enxuta: 13 tecnologias no backend e na interface simples; FastAPI e React só nas fases 7 e 9 (ver Tecnologias).
- Python 3.12+, scikit-learn para todo o ML clássico, PyTorch para deep learning.
- pandas apenas na camada de dados (`core/data/`); o resto do sistema trabalha com NumPy.
- Implementações didáticas em NumPy puro para os algoritmos simples, ao lado das versões do scikit-learn.
- Registo de experiências em SQLite com o módulo `sqlite3` da biblioteca padrão, atrás de um contrato trocável.
- Interface em duas versões com as mesmas funcionalidades: simples (Streamlit) a par de cada fase; aprimorada e personalizada (React) na fase 9.
- Reinforcement learning na fase 8, com ambiente próprio em grelha, sem bibliotecas de RL.
- Execução local, num só utilizador; sem autenticação nem filas distribuídas.

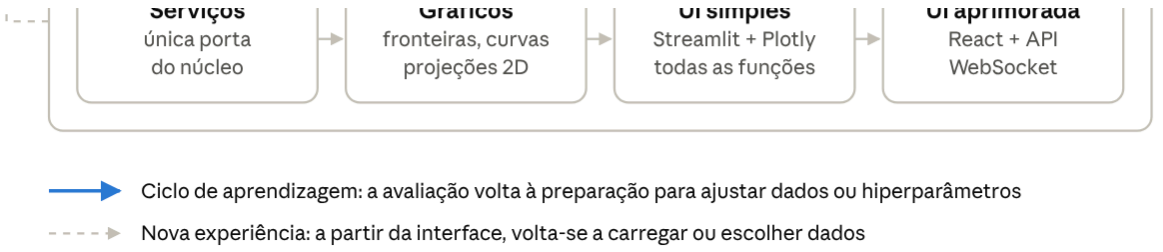
Mapa do projeto

Cada experiência atravessa sete regiões, da entrada dos dados até à interface. Como num cérebro, há dois circuitos de retorno: a avaliação devolve o que aprendeu à preparação, e a interface lança novas experiências.

Sete regiões em sequência; a avaliação realimenta a preparação

Mapa de uma experiência no ML Lab, da entrada de dados à interface





mapa do projeto · 7 regiões, 28 etapas, 2 ciclos

O que cada região faz, em resumo:

Região	Pergunta a que responde	Entrega à região seguinte
1 · Entrada de dados	Que dados tenho e em que estado estão?	Tabela validada, perfilada e limpa
2 · Preparação	O que quero prever e como separo os dados?	`Dataset` em NumPy, dividido e transformado
3 · Modelação	Que algoritmo uso e com que hiperparâmetros?	Modelo treinado e previsões
4 · Observação do treino	O que aconteceu durante o treino?	Sequência de eventos e snapshots
5 · Avaliação	O modelo é bom? Porquê? Onde falha?	Relatório com métricas, diagnósticos e explicações
6 · Memória	Como guardo, comparo e repito?	Run guardado e comparável
7 · Apresentação	Como mostro tudo isto ao utilizador?	Ecrãs da interface

As regiões 3 e 4 acontecem ao mesmo tempo: enquanto o modelo treina, a observação recebe cada passo. A região 4 alimenta também a 6 (os snapshots ficam guardados) e a 7 (o treino aparece em tempo real).

Tecnologias

A stack final tem **13 tecnologias** para o backend e a interface simples, e mais **7** que só entram nas fases 7 e 9 (API e interface aprimorada em React). Todos os tipos de aprendizagem do plano ficam cobertos. Cada tecnologia tem um único papel; a coluna "Fase" diz quando a começas a usar.

Ferramentas de desenvolvimento

Tecnologia	O que é	Para que serve no projeto	Fase
Python 3.12+	Linguagem principal do ecossistema de ML	Todo o backend e a interface simples	0
uv	Gestor de pacotes e ambientes virtuais, muito rápido	Instalar dependências, criar o ambiente, fixar versões (`uv.lock`), correr comandos	0
ruff	Linter e formatador numa só ferramenta	Estilo uniforme e deteção de erros comuns antes de correr o código	0
mypy	Verificador estático de tipos	Garantir que cada implementação respeita os contratos	0
pytest	Framework de testes	Testes unitários, de contrato, de integração e de arquitetura	0

Dados e machine learning

Tecnologia	O que é	Para que serve no projeto	Fase
NumPy	Arrays e álgebra linear rápida	Formato interno dos dados depois da camada de dados; implementações didáticas	1
pandas	Tabelas (DataFrames) com colunas de tipos diferentes	Toda a camada de dados (ver Processamento de dados com pandas)	1
scikit-learn	A biblioteca padrão de ML clássico	Todos os algoritmos clássicos (incluindo gradient boosting), pipelines, divisão, validação cruzada, métricas, importância por permutação e dependência parcial. Traz o joblib, usado para guardar modelos	2
PyTorch	Framework de deep learning com loop de treino explícito	Redes neuronais (MLP, CNN) e o DQN do reinforcement learning	6

Backend

Tecnologia	O que é	Para que serve no projeto	Fase
Pydantic v2	Validação de dados com tipos Python	Hiperparâmetros, configurações de experiência, fichas técnicas; gera os formulários das duas interfaces	2
SQLite (módulo `sqlite3`)	Base de dados num único ficheiro, incluída no Python	Registo de experiências, runs e métricas	5

Configuração e logs usam só a biblioteca padrão do Python (um módulo de configuração e o módulo `logging`), sem dependências extra.

Interface simples

Tecnologia	O que é	Para que serve no projeto	Fase
Streamlit	Transforma scripts Python em aplicações web, sem HTML nem JavaScript	Todos os ecrãs da interface simples, construídos a par do backend	0
Plotly	Gráficos interativos (zoom, hover, animação)	Gráficos da interface simples; o Plotly.js dá os mesmos gráficos no React	1

API e interface aprimorada (fases 7 e 9)

Tecnologia	O que é	Para que serve no projeto	Fase
FastAPI	Framework web assíncrona baseada em Pydantic, com WebSockets incluídos	Expor os serviços por HTTP e transmitir o treino em tempo real; documentação automática em `/docs`	7
Uvicorn	Servidor que corre a aplicação FastAPI	Correr a API localmente	7
React	Biblioteca para interfaces feitas de componentes	Interface aprimorada e personalizada	9
TypeScript	JavaScript com tipos	Evitar erros; tipos gerados a partir da documentação da API	9

Tecnologia	O que é	Para que serve no projeto	Fase
Vite	Ferramenta de desenvolvimento e build para front-end	Servidor local rápido e build final	9
Tailwind CSS	CSS por classes utilitárias	Sistema de design próprio (cores, tipografia, modo escuro)	9
D3.js (+ Plotly.js)	Visualização de baixo nível	Animações feitas à medida: fronteiras a mover-se, redes neuronais, centróides	9

Extensões futuras (fora da stack inicial)

Ficaram de fora para manter a stack pequena. A arquitetura já tem o sítio de cada uma, por isso podem entrar mais tarde sem refazer nada.

Tecnologia	O que acrescentaria	Onde entraria
XGBoost / LightGBM	Boosting usado na indústria, para comparar com o do scikit-learn	Novos modelos em `models/ensemble/`
SHAP	Explicação de previsões individuais	`explainability/`
SQLAlchemy + Alembic	ORM e migrações de base de dados	Nova implementação do registo de experiências
MLflow	Registo de experiências pronto, com interface própria	Nova implementação do registo de experiências
pandera	Validação de DataFrames por esquema	`data/validation/`
UMAP	Redução de dimensionalidade não linear	`models/unsupervised/dimensionality/`
Gymnasium / Stable-Baselines3	Ambientes padrão de RL e algoritmos prontos (PPO)	`rl/`
pre-commit / GitHub Actions	Verificações automáticas antes de cada commit e a cada push	Raiz do repositório

Tecnologia	O que acrescentaria	Onde entraria
Polars	Alternativa mais rápida ao pandas	Só `core/data/`

O TensorFlow/Keras não entra: o PyTorch cobre o deep learning e manter dois frameworks não acrescenta aprendizagem.

Tipos de aprendizagem e algoritmos

O sistema organiza-se por **tipo de tarefa** (`TaskType`). O tipo de tarefa decide que algoritmos aparecem, que métricas se calculam e que snapshots fazem sentido. "Didático" = implementação própria em NumPy/PyTorch, feita para aprender; "sklearn" = envolve a implementação oficial.

Catálogo inicial

Paradigma	Tipo de tarefa	Algoritmos	Implementação	Snapshots durante o treino
Supervisionada	Classificação	Regressão logística	Didático + sklearn	Pesos e fronteira por época, loss
Supervisionada	Classificação	Perceptron	Didático	Fronteira após cada correção
Supervisionada	Classificação	KNN	Didático + sklearn	Vizinhos de cada ponto (sem treino iterativo)
Supervisionada	Classificação	Naive Bayes (Gaussiano)	sklearn	Médias e variâncias por classe
Supervisionada	Classificação	Árvore de decisão	sklearn	Árvore crescendo por profundidade
Supervisionada	Classificação	Random Forest	sklearn	Erro OOB à medida que se juntam árvores

Paradigma	Tipo de tarefa	Algoritmos	Implementação	Snapshots durante o treino
Supervisionada	Classificação	SVM (linear e RBF)	sklearn	Vetores de suporte, margem
Supervisionada	Classificação / Regressão	Gradient Boosting e AdaBoost	sklearn	Previsão e loss árvore a árvore
Supervisionada	Regressão	Regressão linear (gradiente descendente)	Didático	Reta/plano e loss por época, trajetória na superfície de erro
Supervisionada	Regressão	Regressão linear (mínimos quadrados), Ridge, Lasso	sklearn	Coeficientes vs. força da regularização
Supervisionada	Regressão	Regressão polinomial	sklearn (pipeline)	Curva por grau, demonstração de overfitting
Supervisionada	Regressão	Árvore / Random Forest de regressão, SVR	sklearn	Função em degraus a refinar-se
Não supervisionada	Clustering	K-means	Didático + sklearn	Centróides e atribuições por iteração
Não supervisionada	Clustering	DBSCAN	sklearn	Pontos núcleo, fronteira e ruído
Não supervisionada	Clustering	Clustering hierárquico (aglomerativo)	sklearn	Dendrograma, fusões passo a passo
Não supervisionada	Clustering	Gaussian Mixture	sklearn	Elipses por iteração do EM

Paradigma	Tipo de tarefa	Algoritmos	Implementação	Snapshots durante o treino
Não supervisionada	Redução de dimensionalidade	PCA	Didático + sklearn	Componentes, variância explicada
Não supervisionada	Redução de dimensionalidade	t-SNE	sklearn	Embedding ao longo das iterações
Não supervisionada	Deteção de anomalias	Isolation Forest, One-Class SVM, LOF	sklearn	Pontuação de anomalia por ponto
Deep learning	Classificação / Regressão	MLP (rede densa)	PyTorch	Loss, pesos, gradientes, ativações e fronteira por época
Deep learning (extensão)	Classificação de imagem	CNN pequena	PyTorch	Filtros e mapas de ativação
Reinforcement learning (fase 7)	Controlo	Q-learning tabular, DQN	Didático (NumPy) / PyTorch	Recompensa por episódio, tabela Q, política

Pontos de extensão previstos

A arquitetura não precisa de mudanças para receber, mais tarde:

- **Semi-supervisionada** (Label Spreading, Self-Training): novo `TaskType` + avaliador.
- **Séries temporais** (ARIMA, previsão com janelas): novo `TaskType`, divisor de dados temporal e métricas próprias.
- **Sistemas de recomendação** (fatorização de matrizes): novo `TaskType` + avaliador de ranking.
- **NLP básico** (TF-IDF + classificador): nova transformação em `preprocessing`, reaproveita os classificadores existentes.

Regras de deteção automática do tipo de tarefa

1. Sem coluna-alvo → só aparecem clustering, redução de dimensionalidade e deteção de anomalias.
2. Alvo categórico (texto, booleano, ou inteiro com poucos valores distintos) → classificação (binária ou multiclasse).
3. Alvo numérico contínuo → regressão.
4. O utilizador pode sempre forçar o tipo; a deteção é só uma sugestão.

Processamento de dados com pandas

Toda a manipulação de dados vive numa única camada isolada, `core/data/`, e é o único sítio do projeto onde o pandas é usado. Os dados entram como ficheiro, passam por nove etapas e saem como arrays NumPy prontos para os modelos.

O que é o pandas

O pandas é a biblioteca de Python para trabalhar com **dados em tabela**, como uma folha de Excel controlada por código. Está construído por cima do NumPy: cada coluna é, por dentro, um array NumPy, o que o torna rápido. É a ferramenta padrão para ler, explorar e limpar dados antes do machine learning.

Os três objetos que vais usar sempre:

Objeto	O que é	Analogia
DataFrame	A tabela completa: linhas e colunas com nome	Uma folha de cálculo
Series	Uma única coluna, com um tipo de dados	Uma coluna da folha
Index	Os rótulos das linhas (por omissão 0, 1, 2…)	Os números das linhas no Excel

Cada coluna tem um **tipo de dados (dtype)**: inteiro, decimal, texto, categoria, booleano ou data. Grande parte do trabalho de preparação é garantir que cada coluna tem o tipo certo; por exemplo, um preço lido como texto não pode ser usado num modelo.

pandas vs. NumPy: porque usamos os dois

	pandas	NumPy
Forma dos dados	Tabela com colunas de tipos diferentes (texto, números, datas)	Matriz com um só tipo numérico
Nomes	Colunas e linhas com nome	Apenas posições (linha 3, coluna 5)
Valores em falta	Suportados e fáceis de detetar	Exigem cuidado manual
Ponto forte	Ler, explorar, limpar, transformar	Cálculo numérico rápido
Onde entra no projeto	Etapas 1 a 8 da camada de dados	Modelos, treino, avaliação

Os modelos de ML precisam de matrizes numéricas; os dados reais chegam como tabelas mistas. O pandas faz a ponte, e a última etapa da camada converte tudo para NumPy.

Porque isolar o pandas numa só camada

- **Clareza:** sabes sempre onde procurar qualquer operação sobre dados.
- **Fronteira simples:** o resto do sistema recebe sempre o mesmo objeto (`Dataset` com arrays NumPy e metadados), venha o dado de onde vier.
- **Substituição:** trocar o pandas pelo Polars (mais rápido, mais recente) afeta só esta pasta.
- **Testes:** cada etapa testa-se sozinha com tabelas pequenas feitas à mão.
- **Verificação automática:** um teste de arquitetura falha se algum ficheiro fora de `core/data/` importar o pandas.

As nove etapas da camada de dados

Cada etapa tem uma pasta própria (ver Estrutura de pastas) e uma só responsabilidade.

#	Etapa	Pasta	O que faz	Altera os dados?
1	Entrada	`io/`	Lê CSV/Excel, datasets de exemplo ou gera dados sintéticos para um DataFrame	Não
2	Validação	`validation/`	Verifica se o ficheiro é utilizável: vazio, colunas repetidas, tamanho, codificação	Não

#	Etapa	Pasta	O que faz	Altera os dados?
3	Perfil	`profiling/`	Descreve cada coluna: tipo inferido, nulos, valores únicos, estatísticas, distribuição, correlações, desequilíbrio de classes	Não
4	Limpeza	`cleaning/`	Remove duplicados, corrige tipos (texto → número/data), remove colunas inúteis (IDs), trata outliers se pedido	Sim, com registo
5	Tarefa	`task/`	O utilizador escolhe a coluna-alvo; o sistema sugere o tipo de tarefa	Não
6	Divisão	`splitting/`	Separa treino, validação e teste (aleatória, estratificada ou temporal), com seed	Separa
7	Transformação	`transforms/`	Imputação de nulos, codificação de categorias, escala, engenharia de features; aprende os parâmetros só no treino	Sim
8	Projeção	`projection/`	Prepara uma versão 2D (duas colunas ou PCA) para os gráficos	Cópia à parte
9	Saída	`boundary.py`	Converte para arrays NumPy, guarda os nomes das features e entrega o `Dataset`	Converte

Regras da camada de dados

1. **Limpeza antes da divisão, transformação depois.** A limpeza não aprende nada com os dados (remover duplicados é igual em qualquer amostra). A transformação aprende parâmetros (a média para imputar, o desvio para escalar) e por isso só pode ver o treino. Se aprendesse com o teste, o modelo estaria a "espreitar" as respostas: é o *data leakage*, e o projeto vai mostrá-lo como exemplo didático.
2. **Nunca alterar o original.** Cada etapa devolve um DataFrame novo; o ficheiro carregado fica intacto.
3. **Tudo fica registado (linhagem).** Cada operação aplicada ("removidas 12 linhas duplicadas", "coluna idade convertida para número") é guardada em `lineage.py`. A interface mostra este histórico com o antes/depois, e a experiência pode ser

reproduzida exatamente.

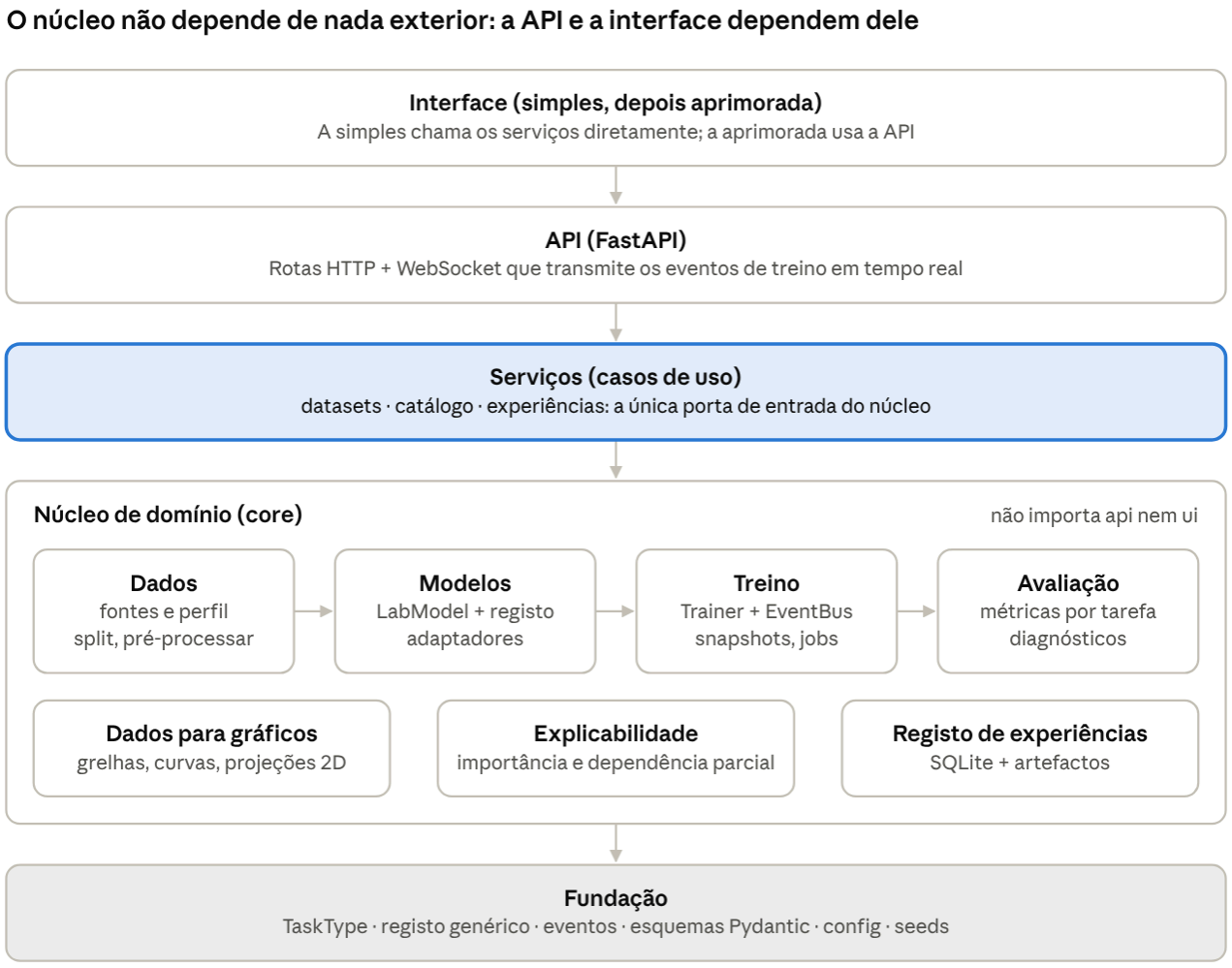
4. **Explicar em vez de esconder.** Cada etapa produz avisos didáticos: "30% de nulos na coluna cabine", "classe positiva só tem 5% das linhas".
5. **Datasets grandes:** o perfil e os gráficos usam uma amostra; o treino usa tudo.
6. **Nomes de colunas preservados até ao fim:** o scikit-learn pode devolver DataFrames nas transformações, por isso os nomes das features chegam à explicabilidade e à interface.

Conceitos de pandas que vais aprender neste projeto

Conceito	Para que serve	Etapa
Leitura de ficheiros (`read_csv` , `read_excel`)	Carregar dados para DataFrame, com separador e codificação certos	1
Tipos (`dtypes` , `astype` , `to_numeric` , `to_datetime`)	Ver e corrigir o tipo de cada coluna	3, 4
Resumos (`describe` , `value_counts` , `nunique`)	Estatísticas e contagens por coluna	3
Valores em falta (`isna` , `fillna` , `dropna`)	Detetar e tratar nulos	3, 4, 7
Correlação (`corr`)	Medir relações entre colunas numéricas	3
Duplicados (`duplicated` , `drop_duplicates`)	Encontrar e remover linhas repetidas	4
Seleção (`loc` , `iloc` , `select_dtypes`)	Escolher linhas e colunas por nome, posição ou tipo	Todas
Agrupamento (`groupby`)	Estatísticas por classe (ex.: média de idade por sobrevivente)	3
Categorias (`category`)	Poupar memória e marcar colunas categóricas	4
Copy-on-Write	Perceber quando uma operação cria uma cópia ou altera o original	Todas
Conversão (`to_numpy`)	Passar da tabela para a matriz numérica	9

Arquitetura em camadas

O sistema tem cinco camadas e as dependências só apontam para baixo: a interface usa os serviços (diretamente na versão simples, através da API na aprimorada), os serviços usam o núcleo, e tudo assenta na fundação.



arquitetura em camadas · as setas indicam "depende de"

Dentro do núcleo, as setas mostram o caminho de uma execução: dados → modelo → treino → avaliação. As três peças de baixo são usadas a pedido.

Fluxo de uma experiência

1. A interface (ou um teste, ou um notebook) pede ao serviço de experiências para correr uma experiência, com uma configuração validada: dataset, preparação, modelo, hiperparâmetros e seed.
2. A camada de dados entrega um `Dataset` já limpo, dividido e transformado (transformações aprendidas só no treino), em NumPy.
3. O serviço vai buscar o modelo ao registo e cria-o com os hiperparâmetros validados.

4. O Trainer abre um run no registo de experiências, anuncia o início e manda o modelo treinar.
5. Durante o treino, o modelo anuncia cada passo. O EventBus entrega cada evento a quem subscreveu: gravador de snapshots, registo de experiências e interface.
6. No fim, o avaliador do tipo de tarefa gera um relatório para treino e para teste; a diferença entre os dois é o alerta de overfitting.
7. Relatório e modelo ficam guardados; o serviço devolve o resultado.
8. A interface pede os dados dos gráficos (fronteira de decisão, curvas, projeção 2D) e desenha-os.

Regra de dependências

Camada	Pode importar	Nunca importa
Interface simples	`services`, `schemas`	Modelos, dados, treino ou base de dados diretamente
Interface aprimorada	API (por HTTP e WebSocket)	Nada do Python diretamente
API	`services`, `schemas`	Modelos, dados, treino ou base de dados diretamente
Serviços	Todo o `core`	`api`, `ui`
Núcleo	Fundação, bibliotecas de ML; pandas só em `core/data/`	`api`, `ui`, `services`
Fundação	Biblioteca padrão, Pydantic	Qualquer outra camada

Estrutura de pastas

O projeto usa o layout `src/` (o código instalável fica em `src/ml_lab/`), que obriga os testes a importar o pacote tal como um utilizador o faria e evita importações acidentais. Cada pasta do núcleo corresponde a uma responsabilidade; cada ficheiro de algoritmo é independente.

```
ml-lab/
├── pyproject.toml              # dependências, grupos opcionais e config de
ruff/mypy/pytest
├── uv.lock                     # versões exatas fixadas (gerado pelo uv)
├── README.md                   # como instalar, correr e contribuir
├── .env.example                # variáveis de ambiente de exemplo
```

```
├── .pre-commit-config.yaml     # verificações antes de cada commit
├── .github/workflows/ci.yml    # testes e lint a cada push
│
├── docs/
│   ├── architecture.md         # este plano, mantido atualizado
│   ├── adr/                    # decisões de arquitetura (uma por ficheiro)
│   └── algorithms/             # uma ficha didática por algoritmo
├── scripts/                    # utilitários: recriar a base de dados,
descarregar exemplos
├── data/
│   ├── samples/                # datasets de exemplo pequenos (versionados)
│   └── uploads/                # datasets carregados (ignorado pelo Git)
├── artifacts/                  # modelos treinados e snapshots (ignorado
pelo Git)
├── notebooks/                  # explorações; nunca importados pelo código
│
├── src/ml_lab/
│   ├── config.py               # definições: caminhos, base de dados, logs
│   ├── logging.py              # configuração de logs estruturados
│   │
│   ├── core/                   # NÚCLEO: não importa api/ nem ui/
│   │   ├── foundation/         # FUNDAÇÃO: peças usadas por todas as camadas
│   │   │   ├── types.py        # TaskType, Paradigm, tipos partilhados
│   │   │   ├── errors.py       # exceções do domínio
│   │   │   ├── registry.py     # registo genérico de componentes (plugins)
│   │   │   ├── events.py       # eventos de treino e EventBus
│   │   │   └── random.py       # gestão central de seeds
│   │   │
│   │   ├── data/               # CAMADA DE DADOS: o único sítio onde o
pandas é usado
│   │   │   ├── README.md       # regras desta camada
│   │   │   ├── dataset.py      # Dataset: o objeto que sai da camada (NumPy
+ metadados)
│   │   │   ├── schema.py       # descrição de colunas: tipo, papel
(feature/alvo/ignorar)
│   │   │   ├── lineage.py      # histórico de todas as operações aplicadas
aos dados
│   │   │   ├── io/             # 1. ENTRADA
│   │   │   │   ├── base.py             # contrato DataSource
│   │   │   │   ├── file_source.py      # CSV e Excel carregados pelo
utilizador
│   │   │   │   ├── builtin_source.py   # Iris, Wine, Titanic, California
Housing
│   │   │   │   └── synthetic_source.py  # moons, blobs, circles, regressão
sintética
│   │   │   ├── validation/     # 2. VALIDAÇÃO
```

```
|   |   |   |   ├── checks.py           # vazio, colunas repetidas, tamanho,
codificação
|   |   |   |   └── schemas.py          # regras esperadas por coluna
|   |   |   ├── profiling/      # 3. PERFIL (só descreve, nunca altera)
|   |   |   |   ├── type_inference.py    # numérica, categórica, booleana,
data, texto, ID
|   |   |   |   ├── column_profile.py    # nulos, únicos, estatísticas por
coluna
|   |   |   |   └── dataset_profile.py   # resumo global, correlações,
desequilíbrio
|   |   |   ├── cleaning/       # 4. LIMPEZA (cada operação fica registada)
|   |   |   |   ├── duplicates.py
|   |   |   |   ├── type_coercion.py     # texto → número, data, booleano
|   |   |   |   ├── columns.py           # remover IDs e colunas inúteis
|   |   |   |   └── outliers.py          # detetar e, se pedido, tratar
extremos
|   |   |   ├── task/           # 5. TAREFA
|   |   |   |   └── task_detection.py    # sugere o TaskType a partir do alvo
|   |   |   ├── splitting/      # 6. DIVISÃO
|   |   |   |   ├── random_split.py
|   |   |   |   ├── stratified_split.py
|   |   |   |   └── time_split.py
|   |   |   ├── transforms/     # 7. TRANSFORMAÇÃO (aprende só com o treino)
|   |   |   |   ├── base.py              # contrato Transform
|   |   |   |   ├── imputation.py
|   |   |   |   ├── encoding.py
|   |   |   |   ├── scaling.py
|   |   |   |   ├── feature_engineering.py  # polinomiais, datas, intervalos
|   |   |   |   └── pipeline_builder.py  # monta o pipeline a partir da
configuração
|   |   |   ├── projection/     # 8. PROJEÇÃO 2D para gráficos
|   |   |   └── boundary.py     # 9. SAÍDA: DataFrame → NumPy (fim do pandas)
|   |   |
|   |   ├── models/             # algoritmos (um ficheiro = um modelo)
|   |   |   ├── base.py         # contrato LabModel + ficha técnica do modelo
|   |   |   ├── adapters/       # ligação ao scikit-learn e ao PyTorch
|   |   |   ├── supervised/
|   |   |   |   ├── classification/   # logística, perceptron, KNN, árvore,
RF, SVM, NB
|   |   |   |   └── regression/       # linear (GD e OLS), Ridge, Lasso,
polinomial, árvores
|   |   |   ├── ensemble/       # gradient boosting, AdaBoost (scikit-learn)
|   |   |   ├── unsupervised/
|   |   |   |   ├── clustering/       # k-means, DBSCAN, hierárquico, GMM
|   |   |   |   ├── dimensionality/   # PCA, t-SNE
|   |   |   |   └── anomaly/          # Isolation Forest, One-Class SVM, LOF
```

```
|   |   |   └── neural/         # redes PyTorch, captura de ativações e
gradientes
|   |   |
|   |   ├── training/           # Trainer, callbacks, snapshots, jobs,
validação cruzada
|   |   ├── evaluation/         # métricas, avaliadores por tarefa,
diagnósticos, explicações
|   |   ├── explainability/     # importância por permutação, dependência
parcial
|   |   ├── visualization_data/ # prepara DADOS para gráficos (sem desenhar)
|   |   ├── tracking/           # registo de experiências, base de dados,
artefactos
|   |   ├── schemas/            # esquemas Pydantic partilhados
|   |   ├── services/           # casos de uso: a única porta de entrada do
núcleo
|   |   └── rl/                 # reservado (fase 8): ambiente em grelha
próprio, Q-learning, DQN
|   |
|   └── api/                    # FastAPI: rotas HTTP e WebSocket
|
├── ui/
|   ├── simple/                 # INTERFACE SIMPLES (Streamlit + Plotly)
|   |   ├── app.py              # entrada e navegação
|   |   ├── pages/              # um ecrã por etapa (Dados, Explorar,
Preparar…)
|   |   ├── components/         # blocos reutilizáveis: cartão de métrica,
formulário
|   |   └── charts/             # desenha com Plotly os dados de
visualization_data
|   └── web/                    # INTERFACE APRIMORADA (React + TypeScript),
fase final
|
└── tests/
    ├── unit/                   # espelha src/ml_lab/core/
    ├── contract/               # aplicados a TODOS os componentes registados
    ├── integration/            # fluxo completo dataset → métricas → base de
dados
    ├── architecture/           # regras de importação entre camadas
    └── api/                    # testes dos endpoints
```

Regras de organização

- `core/data/` é o único sítio onde o pandas é importado; tudo o que sai desta pasta é NumPy dentro de um `Dataset`. Um teste de arquitetura verifica isto.

- As pastas da camada de dados estão numeradas pela ordem das etapas (1 a 9), para leres o processamento de cima para baixo.
- `core/` nunca importa `api/` nem `ui/`.
- `foundation/` não importa nenhuma outra pasta do projeto.
- `visualization_data/` só calcula números (grelhas, curvas, coordenadas). Desenhar é trabalho da interface, para que os mesmos dados sirvam a versão simples e a aprimorada.
- `services/` é a única porta de entrada do núcleo: a interface simples chama-a diretamente; a API chama-a para a interface aprimorada.
- Um algoritmo novo = um ficheiro novo na pasta do seu paradigma. Nenhum ficheiro existente muda.

Contratos centrais

Um contrato é uma promessa: "qualquer componente deste tipo recebe X e devolve Y". O resto do sistema só conhece o contrato, nunca a implementação concreta. É isto que permite adicionar algoritmos sem mexer no código existente. O código vem numa fase seguinte; aqui fica definido o que cada contrato faz.

Visão geral dos contratos

Contrato	Responsabilidade	Recebe	Devolve	Pasta
TaskType	Lista fechada dos tipos de tarefa (classificação binária, multiclasse, regressão, clustering, redução de dimensionalidade, anomalias)	—	—	`foundation/`
Registry	Guardar e encontrar componentes pelo nome; filtrar por tipo de tarefa	Componentes que se registam no arranque	O componente pedido, ou um erro claro	`foundation/`
DataSource	Obter dados de uma origem	Referência (ficheiro, nome de exemplo, parâmetros do gerador)	DataFrame + metadados	`data/io/`

Contrato	Responsabilidade	Recebe	Devolve	Pasta
Transform	Uma transformação de dados que aprende com o treino	Dados de treino (para aprender), depois quaisquer dados	Dados transformados	`data/transforms/`
Dataset	O objeto que sai da camada de dados	—	Arrays NumPy, nomes das features, esquema, linhagem	`data/`
LabModel	Um algoritmo de ML	Hiperparâmetros validados, seed, dados, contexto de treino	Previsões; opcionalmente probabilidades e estado interno	`models/`
TrainingContext	Canal entre o modelo e o exterior durante o treino	Pedidos do modelo para emitir um passo	Eventos publicados; ordem de parar	`training/`
EventBus	Distribuir eventos de treino a quem os subscreve	Eventos	Entrega a cada subscritor	`foundation/`
Evaluator	Avaliar um modelo para um tipo de tarefa	Modelo, dados, partição (treino/teste)	Relatório com métricas, artefactos e avisos	`evaluation/`
Metric	Uma métrica isolada	Valores reais e previstos	Um número + se "maior é melhor"	`evaluation/metrics/`
ExperimentStore	Guardar e consultar experiências	Configuração, eventos, relatórios, modelos	Runs guardados, comparações	`tracking/`

Ficha técnica de cada modelo

Cada modelo descreve-se a si próprio. É esta ficha que permite à interface montar o catálogo, filtrar por tarefa e gerar formulários sem código específico por algoritmo.

Campo	Exemplo (KNN)	Para que serve
Identificador	knn	Nome único no registo
Nome a mostrar	K-Nearest Neighbors	Título na interface
Tipos de tarefa	Classificação binária e multiclasse	Filtrar o catálogo

Campo	Exemplo (KNN)	Para que serve
Família	Baseado em instâncias	Agrupar no catálogo
Implementação	Didática ou scikit-learn	Mostrar a origem; comparar as duas
Iterativo	Não	Saber se emite snapshots passo a passo
Probabilidades	Sim	Permite curva ROC
Hiperparâmetros	número de vizinhos (1 a 100, omissão 5); ponderação (uniforme ou por distância)	Validar e gerar sliders e menus
Ficha didática	docs/algorithms/knn.md	Texto explicativo na interface

Regras de comportamento

1. **O modelo nunca sabe quem o está a observar.** Durante o treino, só comunica através do contexto de treino: anuncia cada passo (época, loss, estado) e pergunta se deve parar.
2. **Capacidades opcionais são declaradas, não adivinhadas.** Se um modelo não dá probabilidades, a ficha diz isso e a interface esconde a curva ROC.
3. **Adaptadores evitam repetição.** Um adaptador para o scikit-learn e outro para o PyTorch implementam o contrato uma vez; cada modelo concreto só indica o estimador, a ficha e os hiperparâmetros.
4. **Métricas são funções puras.** Não guardam estado; o mesmo input dá sempre o mesmo resultado, o que as torna triviais de testar.
5. **Um avaliador por tipo de tarefa.** Recolhe automaticamente todas as métricas registadas para esse tipo; uma métrica nova aparece sozinha nos relatórios.
6. **O registo de experiências é trocável.** Uma versão em memória para testes, uma com SQLite para uso real, e no futuro uma com MLflow, sem mudar quem as usa.
7. **Os snapshots são amostrados.** Guardar o estado a cada passo pode gerar gigabytes; o registo guarda a cada N passos, configurável.

O que vai dentro de cada evento de treino

Evento	Quando acontece	Conteúdo
Início do run	Antes do primeiro passo	Configuração, tamanho dos dados

Evento	Quando acontece	Conteúdo
Passo	Cada época ou iteração	Número do passo, loss, métricas parciais
Snapshot	A cada N passos	Estado do modelo: pesos, centróides, árvore, fronteira
Avaliação	Depois do treino	Relatório de treino e de teste
Fim do run	No final	Duração, resultado, caminho do modelo guardado
Falha	Se ocorrer um erro	Mensagem explicativa

Manutenção: como adicionar componentes

Adicionar qualquer componente segue sempre o mesmo padrão: criar um ficheiro, implementar a interface, registar com o decorador, escrever a ficha didática. Os testes de contrato verificam o novo componente automaticamente.

Adicionar um algoritmo

1. Criar o ficheiro na pasta do paradigma, ex.: `models/supervised/classification/extra_trees.py`.
2. Definir a classe de hiperparâmetros (Pydantic) e a classe do modelo com `info` preenchido.
3. Decorar com `@MODELS.register("extra_trees")`.
4. Escrever `docs/algorithms/extra_trees.md` (intuição, hiperparâmetros, quando usar).
5. Correr `uv run pytest tests/contract` — o modelo é testado sem escrever testes novos.

Adicionar uma métrica

1. Escrever uma função pura em `evaluation/metrics/<tipo>.py`.
2. Decorar com `@metric("balanced_accuracy", task_types={...}, higher_is_better=True)`.
3. Juntar a explicação em `evaluation/explanations.py`.

A métrica aparece em todos os relatórios dos tipos de tarefa indicados.

Adicionar uma fonte de dados

Implementar `DataSource.load() -> Dataset` num novo ficheiro em `data/sources/` e registar em `DATA_SOURCES`. Ex.: uma fonte que lê de uma URL, ou de uma base de dados.

Adicionar um tipo de tarefa (o caso maior)

1. Novo valor em `TaskType`.
2. Regra em `task_detection.py` (se puder ser detetado).
3. Estratégia de divisão em `splitting.py`, se for diferente (séries temporais não podem ser baralhadas).
4. Um `Evaluator` novo + as suas métricas.
5. Funções em `visualization_data/` para os gráficos específicos.
6. Modelos que declaram o novo tipo em `task_types`.

Nenhum código existente muda, exceto o enum e, se necessário, o detetor.

Testes de contrato (a rede de segurança)

Em `tests/contract/`, testes parametrizados percorrem o registo e aplicam a TODOS os componentes:

- Cada modelo treina num dataset mínimo de cada `TaskType` que declara, sem erros.
- `predict` devolve a forma certa; `predict_proba` soma 1 quando existe.
- Mesma seed → mesmo resultado (reprodutibilidade).
- Modelos com `iterative=True` emitem pelo menos um evento `step`.
- Hiperparâmetros por omissão são válidos e o JSON Schema é gerado.
- Cada modelo tem ficha didática no caminho declarado.
- Cada métrica devolve um valor finito em dados de exemplo.

Decisões de arquitetura (ADR)

Cada decisão importante fica num ficheiro curto em `docs/adr/` (ex.: `0001-pytorch-em-vez-de-tensorflow.md`), com contexto, decisão e consequências. Daqui a meses vais saber porque é que algo foi feito assim.

Qualidade e convenções

O objetivo é que qualquer alteração seja verificada automaticamente antes de entrar no repositório: formatação, tipos, testes e regras de arquitetura.

Testes

Nível	Pasta	O que verifica	Quando corre
Unitários	`tests/unit/`	Uma função ou classe isolada (ex.: perfil de dados, uma métrica com valores calculados à mão)	Sempre
Contrato	`tests/contract/`	Todos os componentes registados cumprem o seu contrato	Sempre
Arquitetura	`tests/architecture/`	`core` não importa `api` nem `ui`; pandas só em `core/data/`; `foundation` não importa nada do projeto	Sempre
Integração	`tests/integration/`	Fluxo completo dataset → preparação → treino → avaliação → base de dados	Antes de cada push
API	`tests/api/`	Endpoints HTTP e WebSocket (a partir da fase 7)	Antes de cada push

As implementações didáticas têm um teste extra: o resultado tem de ficar próximo do da versão oficial do scikit-learn no mesmo dataset (ex.: coeficientes da regressão linear iguais até 1e-4). É assim que sabes que a tua implementação está certa.

Convenções de código

- Tipos em todas as funções públicas; `mypy --strict` no pacote `core`.
- Docstrings no estilo Google, em português, nas classes e funções públicas.
- Nomes de código em inglês (padrão do ecossistema); documentação e explicações didáticas em português.
- Sem estado global mutável, exceto os registos (preenchidos só no arranque).
- Erros do domínio com exceções próprias em `core/errors.py`, com mensagens que explicam o que fazer.
- Arrays internos sempre NumPy `float64`; conversão para tensores PyTorch só dentro de `TorchAdapter`.

Git

- Ramo `main` sempre a funcionar; trabalho em ramos `feat/...`, `fix/...`.
- Mensagens de commit no formato Conventional Commits (`feat: adiciona k-means didático`).

- Antes de cada push: `ruff check`, `ruff format`, `mypy` e `pytest`. Automatizar isto (pre-commit, GitHub Actions) fica como extensão.

Documentação

- `README.md`: instalar, correr testes, correr a API.
- `docs/architecture.md`: este plano, atualizado quando a arquitetura mudar.
- `docs/algorithms/<algoritmo>.md`: intuição, matemática essencial, hiperparâmetros, quando usar, armadilhas.
- `docs/adr/`: decisões e as suas razões.
- A API gera documentação interativa automaticamente em `/docs` (Swagger), a partir dos esquemas Pydantic.

Interface

A interface é feita em duas versões com **as mesmas funcionalidades**: primeiro uma simples (Streamlit), que cresce a par do backend em cada fase; no fim, uma aprimorada e bem desenhada (React), que muda a apresentação mas não o que se pode fazer. Assim validas tudo cedo e o redesenho final não obriga a repensar o produto.

Ecrãs (iguais nas duas versões)

Os ecrãs seguem a ordem natural de uma experiência, da esquerda para a direita numa barra de navegação.

#	Ecrã	O que o utilizador faz	O que vê	Fase do backend
1	Dados	Carrega um CSV/Excel, escolhe um dataset de exemplo ou gera um sintético (com controlos de ruído e número de pontos)	Lista de datasets, pré-visualização das primeiras linhas, erros de validação	1
2	Explorar	Navega pelas colunas	Perfil de cada coluna, histogramas, mapa de correlações, mapa de nulos, desequilíbrio de classes, avisos	1

#	Ecrã	O que o utilizador faz	O que vê	Fase do backend
3	Preparar	Escolhe a coluna-alvo, confirma o tipo de tarefa, aplica limpezas e transformações, define a divisão	Antes/depois de cada operação, histórico de passos (linhagem), tamanho de treino/teste	1, 2
4	Modelo	Escolhe um algoritmo do catálogo filtrado pela tarefa e ajusta hiperparâmetros	Cartões por família, ficha didática, formulário gerado da ficha técnica	2, 4
5	Treinar	Inicia, pausa, cancela; percorre o treino passo a passo	Barra de progresso, loss em tempo real, animação dos snapshots com controlos de reprodução, fronteira de decisão / centróides / árvore	3
6	Avaliar	Lê os resultados	Cartões de métricas com explicação, matriz de confusão, ROC e PR, resíduos, silhouette, comparação treino vs. teste, diagnósticos	2, 4
7	Explicar	Escolhe uma previsão ou o modelo inteiro	Importância de features, gráficos de dependência parcial	4
8	Experiências	Filtra o histórico, escolhe 2 ou 3 runs, reproduz um run	Tabela de runs, comparação lado a lado de métricas e gráficos	5
9	Laboratório	Mexe num controlo e vê o efeito imediato	Demos de conceitos: overfitting (complexidade do modelo), taxa de aprendizagem, k do KNN, regularização	4

#	Ecrã	O que o utilizador faz	O que vê	Fase do backend
10	Reinforcement learning	Escolhe ambiente e agente, inicia o treino	Recompensa por episódio, tabela Q, política aprendida	8

Elementos comuns a todos os ecrãs

- Barra lateral com o estado da experiência atual: dataset, tarefa, modelo, último resultado.
- Painel "Porquê?" com a explicação didática do que está no ecrã.
- Avisos destacados quando algo merece atenção (fuga de dados, desequilíbrio, overfitting).
- Botão para guardar a experiência em qualquer momento.

Simples vs. aprimorada

Aspeto	Interface simples	Interface aprimorada
Tecnologia	Streamlit + Plotly (só Python)	React + TypeScript + Vite + Tailwind CSS + Plotly.js/D3
Ligação ao backend	Chama a camada de serviços diretamente, no mesmo processo Python	Através da API (HTTP para pedidos, WebSocket para o treino)
Tempo real	Atualização periódica do ecrã de treino	Snapshots chegam em fluxo contínuo
Animações	Slider para percorrer snapshots	Animações suaves, transições entre estados
Design	Tema padrão do Streamlit com pequenos ajustes	Sistema de design próprio: cores, tipografia, componentes, modo escuro
Objetivo	Validar funcionalidades e aprender depressa	Experiência polida para usar e mostrar
Quando	Cresce a cada fase, a partir da fase 1	Fase 9, com a API estável

Regras para as duas versões partilharem tudo

1. Nenhuma lógica de ML ou de dados vive na interface; a interface só pede e mostra.
2. Os gráficos são desenhados a partir dos dados preparados por `visualization_data/`: a versão simples e a aprimorada recebem exatamente os mesmos números.
3. Os formulários de hiperparâmetros são gerados a partir das fichas técnicas dos modelos, nas duas versões.
4. Os textos didáticos vêm do backend (fichas e explicações de métricas), por isso só se escrevem uma vez.
5. Cada ecrã da versão simples é o rascunho funcional do mesmo ecrã da versão aprimorada.

Roadmap por fases

Dez fases (0 a 9). Cada uma termina com algo que funciona, está testado e já se vê na interface simples. Cada fase introduz poucas tecnologias novas, para as aprenderes uma de cada vez.

Fase	Backend	Interface simples (ecrãs que ganha)	Tecnologias novas	Pronto quando
0 — Fundações	Repositório, configuração, logs, registo genérico, tipos, eventos, seeds; testes de arquitetura	Esqueleto com navegação entre ecrãs vazios	Python, uv, ruff, mypy, pytest, Streamlit	`pytest` e `mypy` passam; a app abre
1 — Camada de dados	As nove etapas: entrada, validação, perfil, limpeza, tarefa, divisão, transformação, projeção, saída; linhagem	Dados, Explorar, Preparar	pandas, NumPy, Plotly	Carregar o Titanic mostra perfil correto, avisos e a sugestão "classificação binária"; o teste de arquitetura confirma o pandas isolado

Fase	Backend	Interface simples (ecrãs que ganha)	Tecnologias novas	Pronto quando
2 — Primeiro fluxo completo	Contrato de modelo, adaptador scikit-learn, regressão logística, avaliador de classificação, serviço de experiências	Modelo, Avaliar (básico)	scikit-learn, Pydantic	No Iris: escolher modelo, treinar e ver métricas na interface; testes de contrato a passar
3 — Eventos e snapshots	Contexto de treino, gravador de snapshots, regressões didáticas em NumPy, dados da fronteira de decisão	Treinar (com reprodução passo a passo)	— (gradiente descendente, padrão observer)	Ver a fronteira da regressão logística didática a mover-se; resultado final igual ao do scikit-learn
4 — Catálogo clássico	Todos os modelos supervisionados e não supervisionados; avaliadores de regressão, clustering e anomalias; diagnósticos; validação cruzada; importância por permutação	Avaliar (completo), Explicar, Laboratório	— (cada algoritmo e as suas métricas)	Todos os modelos passam os testes de contrato e têm ficha didática
5 — Memória	Base de dados, registo de experiências, artefactos, comparação e reprodução de runs	Experiências	SQLite (`sqlite3`), joblib	Fechar a app, reabrir e comparar dois runs

Fase	Backend	Interface simples (ecrãs que ganha)	Tecnologias novas	Pronto quando
6 — Deep learning	Adaptador PyTorch, MLP, captura de ativações e gradientes, early stopping	Treinar mostra as camadas da rede	PyTorch	A MLP resolve "moons" e os snapshots mostram a fronteira a curvar
7 — API	Rotas de dados, catálogo e experiências; treino em background; WebSocket	— (a simples continua a usar os serviços)	FastAPI, Uvicorn	Pela documentação em `/docs`: carregar CSV, lançar treino e receber snapshots
8 — Reinforcement learning (opcional)	Contratos de ambiente, agente e episódio; ambiente em grelha próprio; Q-learning didático; DQN em PyTorch	Reinforcement learning	—	Um agente aprende a atravessar a grelha e o run fica registado
9 — Interface aprimorada	Ajustes à API que a interface pedir	Todos os ecrãs refeitos em React com design próprio	React, TypeScript, Vite, Tailwind CSS, D3.js, Plotly.js	Tudo o que a versão simples faz, com animações suaves e treino em tempo real

Ordem de leitura recomendada para aprender

1. Ler a ficha didática do algoritmo antes de o implementar.
2. Implementar a versão didática (quando existir) e comparar com a do scikit-learn.
3. Só depois ligar à interface e observar os snapshots.
4. Registar uma decisão de arquitetura (ADR) sempre que algo não for óbvio.

Começar a implementar

A implementação começa pela fase 0. No fim dela, o projeto instala-se com um comando, os testes correm, as regras de arquitetura são verificadas e a interface simples abre com a navegação entre ecrãs vazios.

Fase 0, passo a passo

- [ ] Instalar o uv e criar o projeto com o layout `src/ml_lab/`
- [ ] Configurar o `pyproject.toml`: dependências, grupo de desenvolvimento (ruff, mypy, pytest) e as regras de cada ferramenta
- [ ] Criar a árvore de pastas da secção Estrutura de pastas, com ficheiros vazios e um README curto em cada camada
- [ ] `config.py` e `logging.py` com a biblioteca padrão
- [ ] Fundação: tipos de tarefa, exceções do domínio, gestão de seeds
- [ ] Registo genérico de componentes, com testes (registar, obter, filtrar, erro claro em nome desconhecido)
- [ ] EventBus com testes (subscrever, publicar, cancelar subscrição)
- [ ] Testes de arquitetura: `core` não importa `api` / `ui`; pandas só em `core/data/`; `foundation` independente
- [ ] App Streamlit com a navegação entre os ecrãs do plano, todos vazios
- [ ] README com instalação, como correr os testes e como abrir a app
- [ ] Primeiras decisões de arquitetura registadas em `docs/adr/`

Primeiras decisões a registar (ADR)

N.º	Decisão	Porquê
0001	PyTorch em vez de TensorFlow	Loop de treino explícito, ideal para observar o treino passo a passo
0002	pandas isolado em `core/data/`	Fronteira clara entre dados tabulares e cálculo numérico; substituível
0003	Stack enxuta, extensões atrás de contratos	Menos tecnologias para aprender de cada vez, sem fechar portas
0004	SQLite com `sqlite3`	Sem dependências extra; o contrato do registo permite trocar mais tarde
0005	Interface simples (Streamlit) primeiro, React no fim	Validar todas as funcionalidades cedo; o redesenho final só muda a apresentação

Regra para cada fase

Uma fase só termina quando o critério "Pronto quando" do roadmap é cumprido, `ruff`, `mypy` e `pytest` passam, e as fichas didáticas dos algoritmos novos estão escritas.